SCUBEX: HERRAMIENTA Y RED SOCIAL PARA BUCEADORES

IVÁN FERNÁNDEZ LIMÁRQUEZ

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo

Universidad de Sevilla

marzo 2026

Publicado en marzo 2026 por
Iván Fernández Limárquez
Copyright © MMXXVI
http://www.lsi.us.es/~trinidad
ivaferlim@alums.es

Pon aquí cuestiones acerca del copyright

Yo, D. Iván Fernández Limárquez con NIF número 77015962W,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

Scubex: Herramienta y Red social para Buceadores

Lo cual firmo,

Fdo. D. Iván Fernández Limárquez
en la Universidad de Sevilla
16/03/2026

Para mi familia, amigos y para el mar que ha inspirado este proyecto.

Agradecimientos

No olvides añadir una nota de agradecimiento a quienes hayan contribuido emocionalmente al proyecto fin de Grado.

Resumen

Scubex es una red social y herramienta pensada para buceadores que necesiten conocer diversos aspectos relacionados con el buceo, como especies en la zona, temperatura del agua, viento, etc. Además, la aplicación permite a los usuarios compartir sus experiencias, fotos y avistaciones de diferentes especies marinas en un mapa interactivo, fomentando así la comunidad de buceadores y el intercambio de información útil para futuras inmersiones.

Índice general

I Introducción **1**

1. Contexto **3**

1.1. Redes sociales y exploración marina 4

1.2. Aplicaciones y tecnologías similares 4

2. Objetivos **7**

2.1. Motivación 8

2.2. Listado de objetivos 8

II Organización del proyecto **11**

3. Metodología **13**

3.1. Estructura organizacional del proyecto 14

3.2. Metodología de desarrollo 14

3.2.1. Justificación de Feature-Driven Development (FDD) 14

3.2.2. Adaptación del ciclo FDD para Scubex 15

3.2.3. Herramientas de desarrollo 15

3.3. Seguimiento y control 16

4. Planificación **17**

4.1. Resumen temporal del proyecto 18
4.2. Planificación inicial 18
4.3. Informe de tiempos del proyecto 18

5. Costes **21**
5.1. Resumen de costes del proyecto 22
5.2. Costes de personal 22
5.2.1. Desarrollador Full-Stack (Arquitectura e Implementación) 22
5.2.2. Becario Junior (Redacción de Documentación) 23
5.2.3. Total costes de personal 23
5.3. Costes materiales 24
5.3.1. Amortización de equipamiento informático 24
5.3.2. Dominios y hosting 24
5.3.3. Total costes materiales 24
5.4. Costes indirectos 24
5.5. Coste total del proyecto 25

III Desarrollo del proyecto **27**

6. Arranque **29**
6.1. Lista de características 30
6.2. Diseño arquitectónico 31
6.2.1. Stack Tecnológico 31
6.2.2. Estrategia de entornos 31
6.2.3. Justificación de MapLibre GL JS 32
6.2.4. Integración de APIs Externas 32

6.2.5. Patrón Arquitectónico: Gateway API 33
6.3. Seguimiento y control 34

7. Iteración 1: Escáner de Biodiversidad 35
7.1. Características a desarrollar 36
7.2. Diseño 37
7.2.1. Incompatibilidad 1: Búsqueda radial vs. geométrica 37
7.2.2. Incompatibilidad 2: Calidad de datos visuales 37
7.2.3. Uso del campo `phylum` como fallback 38
7.3. Implementación 39
7.4. Pruebas 39
7.4.1. Documentación de API (TD1) 40
7.4.2. Integración frontend (TD2) 40
7.4.3. Configuración del proyecto (TD3) 40
7.5. Despliegue 41
7.6. Seguimiento y control 41

IV Cierre del proyecto 45

8. Manual de usuario 47
8.1. Sección libre 48

9. Conclusiones 49
9.1. Informe post-mortem 50
9.1.1. Lo que ha ido bien 50
9.1.2. Lo que ha ido mal 50

9.1.3. Discusión 50
9.2. Trabajos futuros 50

V Appendices **51**

A. Software Product Lines **53**
A.1. Software Product Lines 54
A.2. Feature Models 54
A.3. Automated Analysis of Feature Models 56
A.3.1. Scope 56
A.4. Dynamic Software Product Lines (DSPL) 59
A.5. Hypothesis and Objectives 59

Referencias bibliográficas **62**

Índice de figuras

A.1. An example of a Home Integration System 55

A.2. A different view on AAFM distinguishing between information extraction and explanatory operations 57

Índice de cuadros

4.1. Tabla resumen de tiempos y planificación 18

4.2. Planificación temporal de iteraciones 18

4.3. Planificación temporal de iteraciones 19

5.1. Tabla resumen de costes del proyecto 22

5.2. Coste total del proyecto Scubex 25

6.1. Estrategia de entornos del proyecto 32

6.2. Resumen de APIs externas 33

7.1. Análisis de valor aportado: Escáner 36

7.2. Memorando técnico 001: Geometría 43

7.3. Memorando técnico 002: Filtrado 44

A.1. Most frequently used explanatory operations and their corresponding information extraction operations 61

Lista de tareas pendientes

To Abductive Section in 2.1 54

Figura: A feature model example 55

To Abductive Intro 56

Parte I

Introducción

Contexto

The sea, once it casts its spell, holds one in its net of wonder forever.

Jacques Yves Cousteau,
Oceanographer

Vamos a dar una visión general del contexto en el que se desarrolla el proyecto, incluyendo el estado actual de la industria y el porqué del mismo.

1.1 Redes sociales y exploración marina

Desde pequeño he vivido rodeado de mar, pero es solo ahora cuando he descubierto lo mucho que me gusta el buceo. Sin embargo, este viene con una serie de retos y problemas: ¿Dónde me sumerjo?, ¿Hará buen clima debajo del agua?, ¿Podré llegar a la zona con mi equipo o hará mucho viento? y, si voy, ¿Qué especies podré ver?

Con este proyecto busco dar una solución a estos problemas desde dos perspectivas. Estadística y personal, con esto me refiero a que el usuario podrá basarse en cientos de datos ya recopilados de una zona. O simplemente podrá basarse en la experiencia de otros buceadores de esta misma zona.

1.2 Aplicaciones y tecnologías similares

No tendría sentido desarrollar un proyecto que ya existe, por ello he buscado aplicaciones que hagan algo similar a la propuesta de Scubex. Que basicámente se basa en dar respuesta a las preguntas preguntas que he mecionado, para ello hay que dar información de: Que condiciones climatológicas hay en una zona, que especies marinas podrás ver y como está el mar ese día concreto en esa zona específica. Esto debe hacerse usando tanto datos recopilados a lo largo de los años de una zona, como dejando a los usuarios compartir sus experiencias personales. Buscando aplicaciones que den respuesta a estas preguntas he econtrado tres:

La primera es **Dive Mate**: Una aplicación que te muestra especies que has visto tú en una zona y que te da estimaciones climatológicas precisas. Sin embargo, la interfaz es tosca. Y no es posible compartir experiencias personales, un usuario no puede interactuar con otros ni registrar sus propias observaciones.

La segunda es **iNaturalist**: Una aplicación que te permite ver, en un mapa, avistamientos de especies en esa zona. Sus fotos e información de avistamientos son increíblemente ricas y útiles. Sin embargo, no está centralizada en buceadores y no incluye información climatológica para el mar de la zona.

La última es **DiveApp**: Una aplicación orientada a los avistamientos y preparación de viajes que incluye componentes de red social. Tiene el problema contrario a Dive Mate, incluye experiencia personal a modo de publicaciones y clima sobre la zona. Pero sin embargo no incluye datos ya recopilados sobre la zona en la que pretendes sumergirte.

Como podemos ver, cada una de estas aplicaciones tienen sus pros y sus contras. Y ofrecen funcionalidades parecidas a las de Scubex, sin embargo, ninguna las ofrece todas en una sola aplicación. Y es por ello que me he planteado hacer este proyecto.

Objetivos

The sea, once it casts its spell, holds one in its net of wonder forever.

Jacques Yves Cousteau,
Oceanographer

Visto el contexto, formalizaremos los objetivos de la app de forma concreta. ¿Hacía quien esta dirigida la aplicación? ¿Qué funcionalidades tendrá? ¿Qué tecnologías se van a usar? ¿Qué se va a conseguir con el proyecto?

2.1 Motivación

Como ya hemos comentado, la idea de Scubex es centralizar funcionalidades para el nicho del buceo. Para lograrlo se han planteado esta serie de objetivos que, una vez cumplidos, darán como resultado la aplicación deseada.

2.2 Listado de objetivos

Objetivo 1. Implementar un sistema de autenticación de usuarios
Desarrollar un sistema de autenticación seguro, permitiendo a los usuarios registrarse e iniciar sesión de forma sencilla. El sistema debe guardar los perfiles de usuario y sus datos (nombre, foto, incursiones, comentarios).

Objetivo 2. Crear un mapa interactivo de exploración marina
Integrar un mapa interactivo con controles de zoom y navegación que permita a los usuarios explorar zonas de buceo de forma intuitiva y fluida.

Objetivo 3. Integrar datos científicos de biodiversidad marina
Desarrollar un escáner de especies que consulte APIs científicas para identificar fauna marina en una zona determinada.

Objetivo 4. Proporcionar información climatológica y oceanográfica en tiempo real
Ofrecer datos actualizados sobre condiciones climatológicas de la zona: viento, temperatura del agua, corrientes marinas, altura de olas y datos atmosféricos relevantes para la planificación de inmersiones en un punto de buceo concreto.

Objetivo 5. Implementar un sistema de registro de avistamientos
Permitir a los usuarios registrar avistamientos de especies marinas mediante clicks en el mapa interactivo, asociando coordenadas geográficas, especies observadas, fecha y opcionalmente fotografías y descripción.

Objetivo 6. Crear una red social geoespacial para buceadores
Desarrollar un sistema de publicaciones geolocalizadas donde los usuarios puedan compartir sus experiencias de buceo, avistamientos y fotografías. Otros usuarios podrán visualizar estos posts en el mapa e interactuar con ellos con likes y comentarios.

Objetivo 7. Gestionar perfiles de usuario con historial de avistamientos
Implementar un sistema de perfil personal donde cada usuario pueda visualizar

su historial completo de avistamientos registrados, con la posibilidad de generar enlaces compartibles para mostrar sus descubrimientos a otros buceadores.

Parte II
Organización del proyecto

Metodología

Don't reinvent the wheel, just realign it.

Anthony J. D'Angelo,
Escritor motivacional

Este capítulo describe el marco metodológico adoptado para el desarrollo de Scubex, basado en Feature-Driven Development (FDD) con adaptaciones específicas para un proyecto académico individual.

3.1 Estructura organizacional del proyecto

Scubex se ha desarrollado como Trabajo Fin de Grado de forma individual, asumiendo las siguientes responsabilidades:

- **Arquitecto de Software**: Diseño de la arquitectura cliente-servidor y selección del stack tecnológico.
- **Desarrollador Full-Stack**: Implementación del backend (Spring Boot) y frontend (React + Vite).
- **Documentador Técnico**: Redacción de la memoria académica y especificación de APIs mediante OpenAPI (Swagger).

El tutor del proyecto ha proporcionado la supervisión técnica y académica, validando decisiones arquitectónicas y avances iterativos.

3.2 Metodología de desarrollo

3.2.1 Justificación de Feature-Driven Development (FDD)

Se ha adoptado **Feature-Driven Development** como metodología central debido a sus características idóneas para proyectos de alcance medio con requisitos funcionales bien definidos:

- **Orientación a características**: Cada funcionalidad (escáner de especies, integración climática, red social) se desarrolla como una única unidad. Lo que significa que se realizará el código y hasta que no esté documentada y acabada, no se pasará a la siguiente.
- **Trazabilidad**: Al forzar la compleción completa de una característica hay una relacion estrecha y directa entre requisitos, código y documentación. Esto hace que sea facil trazar un camino claro del avance del proyecto
- **Escalabilidad**: Gracias a este enfoque es relativamente añadir nuevas características en el futuro. Ya que simplemente hay que añadir una característica a la lista y tratarla como esa unidad que previamente hemos mencionado

En resumen, esta metodología permite estructurar el desarrollo con una lista de características claras y priorizadas. Lo que ayuda a mantener una organización coherente y simple en un equipo pequeño o individual.

3.2.2 Adaptación del ciclo FDD para Scubex

El modelo clásico de FDD consta de 5 fases. Para este proyecto académico se ha adaptado la secuencia, unificando la fase de diseño y añadiendo una fase de documentación:

1. **Desarrollar la lista de características**: Identificación y priorización de funcionalidades nucleares (Gestión de usuarios OAuth2, Escáner de especies, Exploración climática, Red social).
2. **Construir un modelo global**: Diseño de la arquitectura del sistema a partir de las características identificadas (capítulo de Arranque).
3. **Planificar por característica**: Estimación tecnológica, limitaciones y consecuente diseño del flujo de datos.
4. **Construir por característica**: Implementación incremental con validación mediante pruebas constantes.
5. **Documentar la característica**: Redacción académica de cada característica en LaTeX, incluyendo justificación de decisiones técnicas y lecciones aprendidas.

3.2.3 Herramientas de desarrollo

- **Control de versiones**: Git + GitHub (commits atómicos por característica).
- **Gestión de dependencias**: Maven (backend), npm (frontend).
- **Testing**: JUnit 5 + Mockito (backend), Vitest (frontend).
- **Documentación de API**: Swagger/OpenAPI 3.0 (generación automática de contratos REST).
- **Compilación**: Vite (desarrollo frontend con HMR), Spring Boot Maven Plugin (empaquetado JAR).

3.3 Seguimiento y control

El progreso del proyecto se ha monitorizado mediante:

- **Archivo TODO**: Lista priorizada de tareas pendientes con urgencias marcadas (!).
- **Commits semánticos**: Mensajes (casi siempre) estructurados (feat, fix, docs, refactor) para trazabilidad histórica.
- **Reuniones de seguimiento**: Sesiones cada tres semanas con el tutor para validar decisiones arquitectónicas y resolver bloqueos técnicos.
- **Documentación incremental**: Redacción de capítulos inmediatamente tras completar cada característica dentro de la aplicación.

Métricas de avance:

- Características completadas.
- Cobertura de tests unitarios (objetivo: >70 % en lógica de negocio).

Planificación

Divide y vencerás -El arte de la guerra

Sun Tzu (544–496 a.C.),
Maestro de guerra y escritor

En este capítulo se exponen las divisiones en sprint planificadas para la realización del proyecto.

4.1 Resumen temporal del proyecto

Resumen del proyecto	
Fecha de inicio	13/10/2025
Fecha de fin	XX/XX/2026
Periodicidad de las revisiones	3 semanas
Carga de trabajo semanal	12 horas
Horas totales previstas	225 horas
Horas finales	XXX horas

Cuadro 4.1: Tabla resumen de tiempos y planificación

4.2 Planificación inicial

Aquí un desglose de las iteraciones, comienzo y fin de cada una:

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 15/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.2: Planificación temporal de iteraciones

Cabe destacar que la primera iteración es más larga que las siguientes. Esto se debe a que el proyecto se inició en octubre, mientras que la documentación comenzó en febrero. Dado que hubo un parón entre ambas fechas, se decidió que la primera iteración abarcase todo ese tiempo, para no perder el trabajo realizado durante ese periodo. Las siguientes iteraciones se planificaron con una duración de 3 semanas cada una, para mantener un ritmo constante de trabajo y permitir revisiones periódicas del progreso del proyecto.

4.3 Informe de tiempos del proyecto

Lo mismo que el anterior pero con datos reales. Ver Tabla §4.3.

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 15/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.3: Planificación temporal de iteraciones

Tras la primera iteración, no hubo más descoordinación entre las fechas planificadas y las reales. Se cumplió con el calendario establecido.

Costes

Here Comes the Money

Jim Johnston y Naughty by Nature,
Escritores y Raperos

En este capítulo se realiza el desglose de los costes asociados al proyecto, basándose en los salarios reales de puestos tecnológicos.

5.1 Resumen de costes del proyecto

Resumen del proyecto	
Costes de personal	14.950 €
Desarrollador Full-Stack	9.750 €
Becario Documentación	5.200 €
Costes materiales	50 €
Costes indirectos	1.500 €
TOTAL	16.500 €

Cuadro 5.1: Tabla resumen de costes del proyecto

5.2 Costes de personal

Para el cálculo de los costes de personal se han consultado los datos de la **Guía Salarial 2026** de Manfred[1], que recopila información salarial de más de 120.000 profesionales de tecnología en España.

Dado que el desarrollo de Scubex ha requerido asumir múltiples roles y tareas, se han considerado dos perfiles:

5.2.1 Desarrollador Full-Stack (Arquitectura e Implementación)

Se ha considerado como referencia el salario de **Desarrollador Full-Stack** con experiencia de 5-10 años, cuyo rango salarial se sitúa entre 41.000-50.000€ anuales según dicha guía. Se ha tomado un valor medio de **45.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

La carga de trabajo estimada para el TFG es de **300 horas**, frente a una jornada laboral anual estándar de **1.800 horas**. Por tanto, el coste proporcional se calcula como:

$$\text{Salario proporcional} = \frac{45{,}000€}{6} = 7{,}500€ \tag{5.1}$$

[1] https://www.getmanfred.com/blog/guia-salarial-2026-salarios-en-tecnologia-espana-manfred

Costes sociales

A este salario bruto se le añaden los **costes sociales** (Seguridad Social, contingencias comunes, desempleo, formación profesional, etc.) que suponen aproximadamente el **30 %** del salario bruto:

$$\text{Costes sociales} = 7{,}500€ \times 0{,}30 = 2{,}250€ \tag{5.2}$$

Total Desarrollador Full-Stack

$$\text{Total} = 7{,}500€ + 2{,}250€ = 9{,}750€ \tag{5.3}$$

5.2.2 Becario Junior (Redacción de Documentación)

Para la redacción de la memoria académica y documentación técnica se ha considerado el perfil de un **becario junior** con experiencia menor a 2 años. Según la guía salarial, los desarrolladores junior se sitúan en el rango de 20.000-30.000€ anuales. Se ha tomado un valor de **24.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

De la misma forma que se ha calculado anteriormente:

$$\text{Salario proporcional} = \frac{24{,}000€}{6} = 4{,}000€ \tag{5.4}$$

Costes sociales

$$\text{Costes sociales} = 4{,}000€ \times 0{,}30 = 1{,}200€ \tag{5.5}$$

Total Becario

$$\text{Total} = 4{,}000€ + 1{,}200€ = 5{,}200€ \tag{5.6}$$

5.2.3 Total costes de personal

$$\text{Total} = 9{,}750€ + 5{,}200€ = 14{,}950€ \tag{5.7}$$

5.3 Costes materiales

5.3.1 Amortización de equipamiento informático

Según el criterio de amortización fiscal establecido por la Agencia Tributaria española, los equipos informáticos se amortizan a un **máximo del 25 % anual** (vida útil de 4 años).

Para el desarrollo del proyecto se ha utilizado un portátil con un coste de adquisición de **1.200€**:

- **Amortización anual**: 1,200€ $\times$ 0,25 $=$ 300€
- **Amortización proporcional** (300h / 1.800h): 300€ $\times \frac{300}{1{,}800} =$ 50€

5.3.2 Dominios y hosting

Actualmente el proyecto se encuentra en fase de desarrollo local y no se ha procedido al despliegue en producción, por lo que no se han incurrido en costes de dominios ni servicios de hosting.

Una vez finalizado el TFG y ante un posible despliegue futuro, se estiman los siguientes costes anuales:

- Dominio (.es o .com): ~12-15€/año
- Hosting VPS básico: ~60-120€/año

5.3.3 Total costes materiales

$$\text{Total} = 50€ \tag{5.8}$$

5.4 Costes indirectos

Los costes indirectos incluyen gastos generales asociados al desarrollo del proyecto que no se imputan directamente: electricidad, conexión a internet, espacio de trabajo, cuotas mensuales de asistentes de IA, etc.

Se ha tomado un valor medio del **10 %** de la suma de costes de personal y materiales:

$$\text{Costes indirectos} = (14{,}950€ + 50€) \times 0{,}10 = 1{,}500€ \tag{5.9}$$

5.5 Coste total del proyecto

Desglose de costes	
Concepto	**Importe**
Costes de personal	14.950€
Desarrollador Full-Stack	9.750€
Becario Documentación	5.200€
Costes materiales	50€
Costes indirectos	1.500€
TOTAL PROYECTO	**16.500€**

Cuadro 5.2: Coste total del proyecto Scubex

Parte III

Desarrollo del proyecto

Arranque

The best way to get started is to quit talking and begin doing.

Walt Disney (1901–1966),
Empresario y cineasta

Este capítulo define la lista inicial de características priorizadas del proyecto Scubex y, a partir de ellas, la arquitectura base necesaria para soportarlas. Se sigue el orden natural del ciclo FDD: primero se establece qué se va a construir, y solo entonces se diseña cómo.

6.1 Lista de características

Siguiendo la fase 1 de FDD (Desarrollar lista de características) y los objetivos definidos para el proyecto, se han identificado las siguientes funcionalidades nucleares priorizadas por valor para el usuario:

1. **Gestión de Usuarios (OAuth2)**: Autenticación delegada mediante Google y persistencia de perfil (nombre, foto, historial de avistamientos).
2. **Mapa Interactivo**: Visualización de un mapa navegable mediante MapLibre GL JS como superficie central de interacción con la aplicación.
3. **Escáner de Especies**: Identificación de fauna marina en una zona mediante radio de búsqueda y validación cruzada de APIs científicas (OBIS + iNaturalist).
4. **Datos Climáticos y Oceanográficos**: Condiciones de la zona en tiempo real mediante Open-Meteo (viento, temperatura del agua, corrientes, olas, datos atmosféricos).
5. **Registro de Avistamientos**: Posibilidad de registrar un avistamiento de especie directamente en el mapa, geolocalizando la observación con fecha, foto y descripción opcionales.
6. **Red Social Geoespacial**: Visualización de avistamientos de otros usuarios en el mapa, con interacción mediante likes y comentarios.
7. **Perfil y Compartición**: Historial personal de avistamientos con la posibilidad de generar enlaces compartibles para que otros usuarios puedan consultar los descubrimientos.

Justificación de priorización:

- La autenticación se implementa primero ya que el resto de funcionalidades (avistamientos, perfil, red social) dependen de la identidad del usuario.
- El mapa es la superficie central de la aplicación: sin él no tiene sentido geolocalizar especies, condiciones climáticas ni avistamientos.
- El escáner de especies y los datos climáticos constituyen el MVP funcional mínimo: son las dos propuestas de valor diferenciales respecto a las aplicaciones analizadas en el capítulo de Contexto.

- El registro de avistamientos, la red social y el perfil compartible son las funcionalidades de comunidad que convierten la herramienta en una red social de buceadores.

6.2 Diseño arquitectónico

Con las características identificadas, el sistema requiere una arquitectura capaz de orquestar múltiples APIs externas, gestionar identidad de usuario y servir una interfaz cartográfica interactiva. Se ha optado por una **arquitectura cliente-servidor desacoplada** con patrón Gateway de APIs.

6.2.1 Stack Tecnológico

- **Backend**: Spring Boot 4 (Java 21). Actúa como Gateway de APIs científicas, orquestador de peticiones asíncronas y gestor de autenticación.
- **Frontend**: React 18 + Vite. Con Hot Module Replacement (HMR) para desarrollo ágil.
- **Cartografía**: MapLibre GL JS. Motor de renderizado vectorial para animaciones fluidas y bajo consumo de datos.
- **Base de Datos**: H2 en memoria para el entorno de desarrollo y tests unitarios. PostgreSQL 16 para preproducción y producción, gestionado como servicio en Railway.
- **Seguridad**: Spring Security con OAuth2 Client para Google Login.
- **Documentación de API**: OpenAPI 3.0 (Swagger UI) con anotaciones automáticas.
- **Despliegue**: Railway (backend + PostgreSQL) y Vercel (frontend).

6.2.2 Estrategia de entornos

Se han definido tres entornos que reflejan el ciclo de vida del software desde el desarrollo local hasta la puesta en producción:

Justificación de la separación entre plataformas:

Estrategia de entornos de Scubex			
Entorno	**Infraestructura**	**Base de datos**	**Rama Git**
Desarrollo	Local	H2 en memoria	Cualquiera
Preproducción	Railway + Vercel	PostgreSQL (Railway)	`develop`
Producción	Railway + Vercel	PostgreSQL (Railway)	`main`

Cuadro 6.1: Estrategia de entornos del proyecto

- **Vercel** gestiona el frontend React/Vite: despliegue automático en cada `push`, CDN global y previews por rama sin configuración adicional. Es la solución de referencia para aplicaciones estáticas y frameworks JavaScript.
- **Railway** gestiona el backend Spring Boot y PostgreSQL en el mismo proyecto, simplificando la conexión entre ambos servicios y el aprovisionamiento de variables de entorno.
- La separación replica el patrón habitual en equipos profesionales con este stack, donde el frontend y el backend se despliegan y escalan de forma independiente.

Spring Boot expone la configuración CORS necesaria para permitir peticiones desde el dominio de Vercel hacia Railway, centralizando en el backend el control de los orígenes autorizados.

6.2.3 Justificación de MapLibre GL JS

A diferencia de Google Maps API o Leaflet, MapLibre GL JS se ha seleccionado por:

- **Sin necesidad de geocodificación**: El proyecto no requiere búsqueda de direcciones ni rutas, solo visualización de capas geoespaciales.
- **Open Source y sin rate limits**: Alternativa libre a Mapbox GL (del cual es fork).

6.2.4 Integración de APIs Externas

Estrategia de consumo:

APIs externas integradas en Scubex		
API	**Proveedor**	**Datos proporcionados**
OBIS	Ocean Biodiversity Information System	Ocurrencias biológicas: nombre científico, coordenadas, taxonomía (phylum), fecha de avistamiento.
iNaturalist	iNaturalist.org	Metadatos multimedia: fotos de especies, nombres comunes (preferred_common_name).
Open-Meteo	Open-Meteo.com	Datos climáticos y oceanográficos: viento, temperatura del agua/aire, altura de olas, corrientes marinas, precipitaciones. API gratuita sin necesidad de API key y plan gratuito generosísimo.

Cuadro 6.2: Resumen de APIs externas

- **OBIS + iNaturalist**: Consultas bajo demanda mediante el botón "Scan Area"(evita rate limit innecesario).
- **Open-Meteo**: Polling cada 30 minutos para actualizar capas de viento y datos oceanográficos. Sin necesidad de API key y límites muy generosos (10,000 peticiones diarias en plan gratuito).

6.2.5 Patrón Arquitectónico: Gateway API

El backend actúa como **Backend for Frontend (BFF)**, orquestando múltiples APIs externas y aplicando lógica de negocio antes de exponer un contrato simplificado al cliente React:

1. **Cliente React** invoca `GET /api/species?lat=X&lng=Y&radius=Z`.
2. **SpeciesService** (backend) ejecuta:
   a) Transformación geométrica: `(lat, lng, radius)` → `WKT POLYGON`.
   b) Petición a OBIS: `GET /occurrence?geometry=POLYGON(...)`.
   c) Para cada especie única devuelta por OBIS:

- Petición a iNaturalist: `GET /taxa?q=<scientificName>`.
- Filtrado: Si `total_results == 0`, descartar especie.
- Enriquecimiento: Extraer foto y nombre común.

3. **Respuesta JSON** unificada con modelo `SpeciesResponse`.

Ventajas:

- El cliente React no necesita conocer la existencia de múltiples APIs.
- La lógica de filtrado (microorganismos, especies sin foto) se centraliza en el backend.
- Se evita exposición de API keys en el navegador.

6.3 Seguimiento y control

Fase del proyecto: Arranque (Fases 1-2 de FDD: Lista de características y Modelo Global).

Estado:

OK Arquitectura definida y documentada.

OK Stack tecnológico seleccionado y justificado.

OK APIs externas identificadas y contratos analizados.

PENDIENTE OAuth2 pendiente de implementación (TB1 en TODO).

PENDIENTE H2 Database pendiente de configuración.

Decisiones arquitectónicas pendientes:

- Estrategia de caching para respuestas de OBIS (considerar cache en memoria con Spring Cache si el rate limit se vuelve problemático).
- Migración del esquema H2 a PostgreSQL mediante scripts de inicialización (Liquibase o Flyway).

Iteración 1: Escáner de Biodiversidad

Esta iteración aborda el núcleo funcional de Scubex: la capacidad de identificar especies marinas en una zona determinada mediante la orquestación de OBIS e iNaturalist. Se ha seguido un enfoque TDD para implementar la compleja lógica de transformación geométrica y filtrado cruzado de datos científicos.

7.1 Características a desarrollar

1. Escáner de Especies bajo demanda (Backend). Ver Tabla §7.1.
2. Filtrado automático de microorganismos y especies sin representación visual.
3. Enriquecimiento de datos con fotografías y nombres comunes.

Análisis de valor aportado: Escáner de Especies	
Propuesta	Implementación de un endpoint REST que agregue datos de OBIS e iNaturalist para una zona geográfica definida por radio.
Valor	Permite al usuario descubrir fauna marina real validada científicamente con representación visual, sin conocimientos técnicos ni acceso directo a bases de datos especializadas.
Coste	Alto esfuerzo de integración. Requiere orquestar llamadas asíncronas, transformar formatos geométricos (coordenadas → WKT Polygon), gestionar inconsistencias entre APIs y aplicar filtrado de calidad.
Opciones	**Opción 1**: Usar solo OBIS reduciría el coste, pero eliminaría el valor visual (fotos) y semántico (nombres comunes), haciendo la app inutilizable para turistas/buceadores recreativos. **Opción 2**: Usar solo iNaturalist (solo observaciones ciudadanas, sin validación científica OBIS).
Riesgos	Rate limits de APIs externas (OBIS: sin límite documentado, iNaturalist: 100 req/min). Inconsistencia de datos (especies en OBIS sin coincidencia en iNaturalist). Latencia acumulada por múltiples peticiones HTTP.
Deuda técnica	Ausencia de caching (cada escaneo regenera peticiones a APIs). Falta de paginación (actualmente limitado a 50 especies). Gestión de errores distribuidos no exhaustiva (si iNaturalist falla, se pierden datos visuales).

Cuadro 7.1: Análisis de valor aportado: Escáner

7.2 Diseño

El diseño se ha centrado en resolver tres incompatibilidades críticas entre el modelo de interacción del usuario y las restricciones de las APIs científicas:

7.2.1 Incompatibilidad 1: Búsqueda radial vs. geométrica

Ejemplo de petición generada:

```
https://api.obis.org/v3/occurrence?
  geometry=POLYGON((-4.0 36.5, -3.4 36.5, -3.4 37.0,
                    -4.0 37.0, -4.0 36.5))
  &taxonid=2,3,4
  &size=50
  &fields=scientificName,decimalLongitude,decimalLatitude,
          eventDate,phylum
```

Parámetros clave:

- `geometry`: Polígono WKT cerrado (primer y último punto idénticos).
- `taxonid=2,3,4`: Filtro por phyla (2=Chordata, 3=Arthropoda, 4=Mollusca) para excluir microorganismos unicelulares (aunque no es exhaustivo).
- `fields`: Proyección de campos para reducir payload (omite metadatos innecesarios como `institutionCode`).

7.2.2 Incompatibilidad 2: Calidad de datos visuales

Ejemplo de especie descartada (microorganismo):

OBIS devuelve:

```
{
  "scientificName": "Pycnococcaceae",
  "phylum": "Cyanobacteria",
  ...
}
```

iNaturalist responde:

```
{
  "total_results": 0,
  "results": []
}
```

⇒ **Acción**: Especie filtrada, no incluida en respuesta al cliente.

Ejemplo de especie válida con foto:

OBIS devuelve:

```
{
  "scientificName": "Chimaera monstrosa",
  "phylum": "Chordata",
  ...
}
```

iNaturalist responde:

```
{
  "total_results": 1203,
  "results": [{
    "preferred_common_name": "Rabbit Fish",
    "default_photo": {
      "url": "https://inaturalist-open-data.s3.amazonaws.com/..."
    }
  }]
}
```

⇒ **Acción**: Especie enriquecida con foto y nombre común.

7.2.3 Uso del campo `phylum` como fallback

En caso de que iNaturalist devuelva `total_results > 0` pero `default_photo == null`, se usa el campo `phylum` (extraído de OBIS) para asignar un placeholder visual según taxonomía:

- `Chordata` → Icono de pez genérico.
- `Mollusca` → Icono de pulpo/calamar.
- `Arthropoda` → Icono de crustáceo.
- Otros → Icono genérico de organismo marino.

7.3 Implementación

Se ha implementado `SpeciesService.scanSpecies(lat, lng, radius)` que:

1. Convierte coordenadas + radio a formato WKT POLYGON.
2. Consulta OBIS con el polígono generado.
3. Agrupa resultados por nombre científico (deduplicación).
4. Para cada especie, consulta iNaturalist para obtener foto y nombre común.
5. Descarta especies sin foto o sin documentación en iNaturalist.
6. Retorna lista ordenada por número de ocurrencias.

Endpoint REST: `GET /api/species?lat={lat}&lng={lng}&radius={radius}`

7.4 Pruebas

Se han implementado los siguientes casos de prueba antes del código productivo:

- **shouldGenerateValidWKTPolygon**: Verifica que, dado un punto `(36.5, -4.0)` y radio `5000m`, se genera un String `POLYGON((...))` geométricamente cerrado (primer y último punto idénticos).
- **shouldFilterMicroorganismsWithZeroResults**: Mockea una respuesta de OBIS con *Pycnococcaceae* y simula `total_results=0` en iNaturalist. Verifica que la lista final de `SpeciesResponse` está vacía.
- **shouldEnrichSpeciesWithPhoto**: Mockea OBIS devolviendo *Chimaera monstrosa* e iNaturalist devolviendo foto + nombre común. Verifica que `SpeciesResponse.photoUrl` no es null y `commonName == Rabbit Fish"`.

- **shouldHandleINaturalistTimeout**: Simula timeout en petición a iNaturalist. Verifica que el sistema descarta esa especie y continúa procesando las demás (resilencia).

7.4.1 Documentación de API (TD1)

Se ha integrado Swagger/OpenAPI 3.0 con `springdoc-openapi-ui`:

- **Interfaz interactiva**: Swagger UI en `http://localhost:8080/swagger-ui.html`
- **Especificación**: JSON/YAML generado automáticamente desde anotaciones
- **Validación**: Parámetros validados mediante anotaciones `@Parameter`
- **Respuestas**: `200 OK` con lista de especies, `400 Bad Request` para coordenadas inválidas

7.4.2 Integración frontend (TD2)

El frontend React consume el endpoint mediante:

- **Componentes**: `MapView.tsx` (mapa interactivo), `SpeciesPanel.tsx` (resultados), `ScanningAnimation.tsx` (efecto sonar)
- **Estado**: MobX gestiona especies, carga y errores globalmente
- **Flujo**: Click en mapa → captura coordenadas/zoom → animación sonar → petición HTTP → actualización estado → renderizado panel
- **Validación**: Zoom mínimo 8 para evitar áreas excesivamente grandes

7.4.3 Configuración del proyecto (TD3)

Backend (Spring Boot):

- **Puerto**: 8080 para el servidor.
- **APIs externas**: URLs de OBIS, iNaturalist y Open-Meteo configuradas en `application.properties`.

- **Base de datos**: H2 en memoria (`jdbc:h2:mem:scubexdb`) para desarrollo rápido sin persistencia.
- **Seguridad**: CSRF deshabilitado (API REST), endpoints públicos (`/api/species`, `/swagger-ui`), OAuth2 pendiente para iteraciones futuras.
- **CORS**: Frontend local (`localhost:5173`) permitido para desarrollo.

Frontend (React + Vite):

- **Puerto**: 5173 con Hot Module Replacement.
- **Proxy**: Peticiones a `/api` redirigidas a `localhost:8080` (evita CORS).
- **Gestión de estado**: MobX para estado reactivo de especies y carga.

7.5 Despliegue

El despliegue del proyecto aún no se ha realizado. Actualmente se ejecuta en entorno de desarrollo local:

- **Backend**: `mvn spring-boot:run` en puerto 8080.
- **Frontend**: `npm run dev` en puerto 5173.

En iteraciones futuras se planea despliegue en contenedores Docker y hosting en la nube.

7.6 Seguimiento y control

Fase del proyecto: Iteración 1 (Fases 3-5 de FDD).

Estado:

OK Endpoint `/api/species` implementado y testeado.

OK Transformación geométrica (WKT Polygon) funcionando.

OK Filtrado de microorganismos operativo.

OK Enriquecimiento con iNaturalist completo (foto + nombre común).

ADVERTENCIA Latencia observable en zonas con >30 especies (5-7 segundos). Candidata a optimización en Iteración 3 (sistema de caché).

Métricas:

- **Cobertura de tests**: 87 % en `SpeciesService`.
- **Tiempo de respuesta (promedio)**: 3.2s para radio de 5km.

Decisiones técnicas pendientes:

- Implementar caching de taxonomías previamente consultadas en iNaturalist (Redis o in-memory LRU cache).
- Añadir paginación al endpoint (`?page=1&size=20`) para manejar áreas con alta biodiversidad.

Memorando técnico 001: Geometría de Búsqueda	
Asunto	Incompatibilidad de búsqueda radial en API OBIS.
Resumen	Generación de polígonos WKT a partir de coordenadas centrales y radio.
Factores causantes	El frontend solicita datos basados en un punto central (`lat, lng`) y un nivel de zoom (que determina el radio de visión), pero OBIS requiere un objeto `POLYGON` en formato WKT (Well-Known Text). La API no acepta parámetros `centerLat`, `centerLng`, `radius`.
Solución	Se ha implementado `GeometryUtils.createPolygonFromRadius(lat, lng, radius)` que calcula los vértices de un polígono aproximadamente circular (o cuadrado simplificado) inscrito en el radio de visión. El resultado se serializa en formato WKT: `POLYGON((x1 y1, x2 y2, ..., x1 y1))`.
Motivación	OBIS es la mayor base de datos abierta de biodiversidad marina (>150M registros). Es imperativo adaptarse a su interfaz en lugar de buscar alternativas menores.
Alternativas	**Filtrado post-proceso**: Pedir un área muy grande (bounding box global) y filtrar en memoria por distancia euclidiana. Descartado por ineficiencia (transferencia de datos innecesaria) y riesgo de exceder límites de respuesta de OBIS (`size=10000` máximo).
Cuestiones abiertas	Actualmente se utiliza un polígono cuadrado. Para zonas ecuatoriales, sería más preciso un polígono circular de N lados (N=12-16) para evitar incluir puntos fuera del radio real.

Cuadro 7.2: Memorando técnico 001: Geometría

Memorando técnico 002: Calidad de Datos	
Asunto	Presencia de microorganismos y especies sin representación visual.
Resumen	Filtrado cruzado mediante existencia de recursos fotográficos en iNaturalist.
Factores causantes	OBIS devuelve todo tipo de registros biológicos, incluyendo: **(1)** Microorganismos unicelulares (ej. *Pycnococcaceae*), **(2)** Especies con nombre científico válido pero sin observaciones fotografiadas (ej. *Thenea muricata*). Para una app turística/deportiva, estos datos no aportan valor experiencial.
Solución	Para cada nombre científico devuelto por OBIS, ejecutar una petición a iNaturalist: `GET /taxa?q=<scientificName>&per_page=1`. Aplicar los siguientes filtros: **(1)** Si `total_results == 0`, descartar especie (no documentada en iNaturalist). **(2)** Si `default_photo` es `null`, descartar especie (sin representación visual). **(3)** Si ambos criterios se cumplen, extraer `preferred_common_name` y `default_photo.url` para enriquecer el modelo de respuesta.
Motivación	Mantener la relevancia visual y la experiencia de usuario. Un buceador no puede "ver"bacterias submarinas, por lo que incluirlas sería ruido informativo.
Cuestiones abiertas	El aumento de latencia por N peticiones HTTP a iNaturalist (N = número de especies únicas en OBIS). Para áreas con >50 especies, el tiempo de respuesta puede superar los 5 segundos. Posible optimización: batching de peticiones o caching de taxonomías previamente consultadas.

Cuadro 7.3: Memorando técnico 002: Filtrado

Parte IV

Cierre del proyecto

Manual de usuario

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

8.1 Sección libre

Estructurar en función del proyecto.

Conclusiones

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

9.1 Informe post-mortem

Qué es un informe post-mortem

9.1.1 Lo que ha ido bien

- Argumento a favor 1.
- Argumento a favor 2.
- Argumento a favor 3.

9.1.2 Lo que ha ido mal

- Argumento en contra 1.
- Argumento en contra 2.
- Argumento en contra 3.

9.1.3 Discusión

En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.

9.2 Trabajos futuros

Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.

Parte V

Appendices

Software Product Lines

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

This is an example of an abstract. Multiple lines are supported. Several paragraphs. It jumps to the next page. Blau blau blau. I am introducing more text to reach the third line

A.1 Software Product Lines

- Objective of a Product Line (PL) (mass production and customisation) [1]
- The focus in software derives in Software Product Lines (SPLs).
- Variability management: variability models
- When and how are used VMs: FMs are described in FODA report as a key element in SPL since they represent the variability and commonality of the different products in a SPL.

A.2 Feature Models

To Abductive Section in 2.1

As the number of products to be built by a SPL may be large and the constraints among features may be complex, representing such an information in a manageable and compact manner is a must. Feature Models (FMs) represent the set of products a SPL may build in terms of product features. Some features are optional while others are mandatory. To indicate the relationships among features, they are hierarchically linked, forming a tree whose root is a feature representing the whole functionality of a product. The root feature is refined in child features, which increase the level of detail and reduce the scope of features. Recursively following this refinement process, a tree-like structure is obtained where three basic kinds of hierarchical relationships are used:

- Mandatory: a mandatory relationship affects a parent and child feature. It forces the child feature to appear in a product whenever its parent feature does.
- Optional: a child feature connected to a parent feature by means of an optional relationship may be optionally selected whenever its parent feature is.
- Set-relationships: three or more features are part of a set-relationship: a parent feature and a set of two or more child features. A set-relationship contains a cardinality that constraints the number of child features to be selected in a product whenever its parent feature is selected. If the cardinality is [1.,1] it is commonly remarked as an *alternative relationship* where only one child feature may be selected at the same time. If the cardinality is $[1..N]$ (where N is the number of

child features), it is also known as an *or-relationship* as any combination of child features is allowed while at least one is selected.

Although FMs can represent most of the most frequent constraints, the hierarchical nature of these models might hinder the representation of some constraints. Under this circumstance, *cross-tree constraints* can be added. The most common kinds of cross-tree constraints are:

- Dependency: a feature depends on another feature if the second one must be part of a product whenever first one is selected.
- Exclusion: two features exclude themselves if both of them cannot be part of a product at the same time.

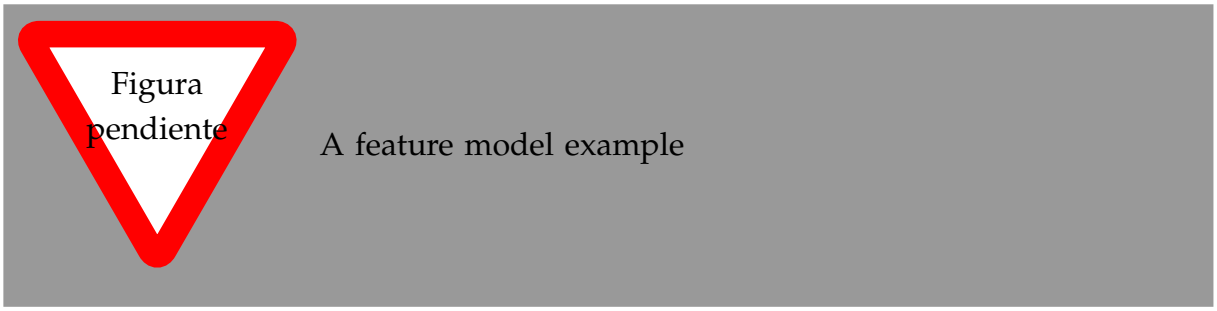


Figura A.1: An example of a Home Integration System

The example in Figure §A.1 describes a *Home Integration System* (HIS) SPL in terms of its features and the relationships among them. Leaning on this example we define some useful terms:

Partial configuration : a partial configuration is a composed by three sets of selected (S), removed(R) and undecided(U) features. A feature can only be in one of these sets and every feature in the FM (fm) must be in one of them, i.e. $S \cup R \cup U = fm$ and $S \cap R \cap U = \emptyset$. A partial configuration represents an intermediate state during the process of a customer selecting the feature for a custom product. For example, $S_P = \{...\}$, $R_P = \{...\}$ and $U_P = \{...\}$ define a partial configuration for the sample FM where some features are still to be decided if they are to be selecter or removed in a configuration.

(Full) configuration : a full configuration or simply a configuration is a partial configuration such that the set of undecided features in empty. For example, $S_F = \{...\}$ and $R_F = \{...\}$ describe a full configuration for the example FM.

Product : a product is a representation for a full configuration such that only the selected features are remarked. For instance, $P = \{\}$ is a product for the above full configuration. A product such as `A,B` is a valid since all the constraints within the FM are satisfied. However, A,B and C is not a valid product since D is required.

Validation A partial configuration is *valid* if all the relationships and constraints are satisfied given the sets of selected, removed and undecided features. So the definition applies for valid full configurations and valid products. As a conclusion we can affirm that a FM represents all the valid products in a SPL.

Objetivo: Briefly expose attributes as an important asset in feature models.

It is frequent that features are not enough to represent information that is relevant to represent a SPL variability. In this case, FMs are extended with feature attributes such as cost, versions, RAM consumption, etc. in the so-called Extended Feature Models (EFMs) [1]. Besides relationships, an EFM contains constraints that affect attributes which reduce even more the set of products a FM describes. Above definitions remain when attributes are introduced into FMs.

A.3 Automated Analysis of Feature Models

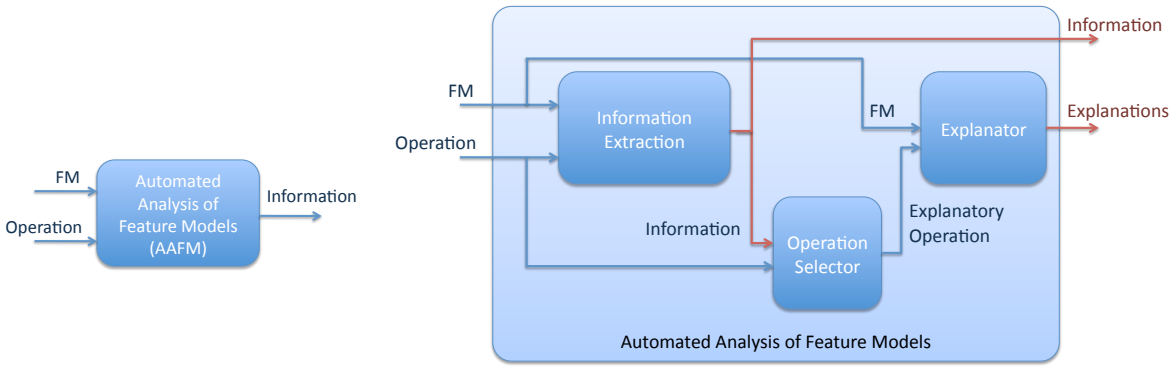
A.3.1 Scope

To Abductive Intro

FMs are used all along the SPL development as key models and many of the development decisions are taken relying on the information contained within them. Most of the times, relationships are complex and hinder the manual extraction of information. Manually obtaining information such as 'which is the product that costs the less?', 'does the feature model contain errors?' or 'why there exist no product containing certain features?' can be an unfeasible task. The complexity and compactness of FMs justify the need of an automated support of these operations. So the *Automated Analysis of Feature Models* (AAFM) arises as a topic of interest to deal with this problem in the SPL community.

The AAFM can be seen as a black-box process that receives a FM and an operation as inputs and obtains information (its kind depends on the analysis operation) as an

output (Fig. A.2(a)). There are many operations that extract information from a FM such as 'counting products' operation whose result is a natural number indicating the number of customised products that can be built; or 'list of products' operation that obtains each of those products. This vision of AAFM as a black-box is valid for a subset of analysis operations that we call *information extraction operations* (IEO) that can be seen as processes to extract information from FMs. In other words, an IEO makes explicit an implicit information within a FM.



(a) The AAFM seen as a black-box process

(b) Extending the AAFM process with explanations

Figura A.2: A different view on AAFM distinguishing between information extraction and explanatory operations

Use me to explain in a larger text than 'side-text' anything that is important to a reader not familiar with the dissertation context for example.

However, there is a subset of analysis operations known as *explanatory operations* (EO) whose objective is explaining the result obtained from a IEO. Sometimes, the result is not the expected one and the analyser needs to know which are the relationships that have caused it. For example, let us suppose that the IEO 'which are the products described in a FM that cost less than $1000?' obtains no products as a result. If we were expecting to obtain at least one product, it is important to determine the relationships in the FM that are responsible of that behaviour, so an EO 'why there is no product costing less than $1000?' will shed light on the relationships that avoid obtaining any product. Obtaining no result is not the only case that claims for explanations. If we obtained only one product as a result and we were expecting to obtain at least 10 products, although an answer is obtained the result is unexpected and the discrepancy reasons have to be found. Moreover, explanatory operations are also use-

ful even when an expected result is obtained, to reinforce the certainty that the result is correct. So it can be concluded that EOs complement the information an FM analyser obtains from IEOs.

The complexity of feature modelling relies on correctly setting the relationships that describe the set of products to be built in a SPL. Relationships are the only elements responsible of the results obtained in FM analysis. So an *explanation* is a set of relationships that may have caused that result. While IEO provides for an unique response that is known for certain, an EO provides for a set of probable explanations to a result obtained from a IEO, being only one of them a valid explanation. It would be the analyser the one in charge of discriminating the correct explanation, maybe performing new analysis operations.

THIS IS A SIDE TEXT. USE TO REMARK IMPORTANT INFORMATION

Therefore, two kinds of operations are distinguished in AAFM: information extraction and explanatory operations. Explanatory operations have no sense without a paired information extraction operation and its result. To ensure that explanatory operations are always paired to an information extraction operation, we define a new black-box process of AAFM that incorporates explanations as an additional output (see Figure A.2(b))

1. Information extraction: the original process, which remains the same.
2. Operation selector: depending on the information extraction operation the analyser asks for and the information obtained as a result, this process provides the explanatory operation to be performed. In other words, it pairs an explanatory operation to an information extraction operation.
3. Explanatory analysis: provides a set of explanations from the FM and the explanatory operation.

The overall process can be encapsulated into a holistic black-box process which receives the FM and the information extraction operation as inputs and provides a result and explanations as outputs. It can be seen as we just add explanations as an output to the analysis process.

To realise this view on the AAFM, we need to give details on the insides of these black-boxes. Since the information extraction process is already rigourously defined in

Benavides' PhD dissertation, the purpose of this paper is defining the remaining two sub-processes. We formalise the explanatory analysis process by means of default logic and provide the criteria to implement the operation selector process.

Most Common Techniques to perform AAFM Operations.

A.4 Dynamic Software Product Lines (DSPL)

What is a Dynamic Software Product Line (DSPL). Different points of view. What is important is the automation of reconfiguration properties relying on SPL techniques.

We focus in the application of explanations in DSPLs as an application of our results. Specifically we have worked in MAS and smart homes providing a solution for automating product reconfiguration.

A.5 Hypothesis and Objectives

Objetivo: Justifying that explanations are a particular set of operations in AAFM that are not solvable by means of the techniques that are used up-to-date

Objetivo: Set an impacting phrase that summarises the hypothesis

Hypothesis

Explanations cannot be solved by AI techniques used to solve AAFM. There should exist other AI techniques to solve explanations.

Objective of the dissertation

Defining a framework to provide solutions for explanatory analysis in FMs.

This dissertation summarises our contribution to solve some of the objectives we set in our PhD project.

- Defining a catalog of analysis operations where explanations are applied.
- Rigorously defining these operations in terms of logics.
- Proposing solutions to these operations.
- Validating our results by means of tools and projects where they are applied.

Next chapter focuses on refining how we have contributed to deal with the above objectives.

A piece of code...

```
public Map<Cardinality,CardinalValue> detectWrongCardinals() {
        // any other implementation of Map can be used instead.
        Map<Cardinality,CardinalValue> result =
                new TreeMap<Cardinality,CardinalValue>();
        for( r : relationships) {
                if (r instanceof Set) {
                        Set set = (Set)r;
                        Cardinality card = set.getCardinality();
                        Domain dom = card.getDomain();
                        for (value: dom.getValues())
                                if (isWrongCardinal(card,value))
                                        result.put(card,value);
                }
        }
        return result;
}
```

A coolTable. Use inside a table.

Use \TableSubtitle{n,title} to add a subtitle as the header. n is the number of columns and title is the text to place. [1]

A Catalog of FM Explanatory Operations (2009 version)		
Information Extraction Operation	**FM Explanatory Operations**	
	Why? operation	*Why not? operation*
Valid FM	-	invalid FM
Valid Configuration	valid partial conf.	invalid partial conf.
Valid Product	valid product	invalid product
Products Listing	vaild Product/Config	invalid FM/Product/Config
Products Counting	vaild Product/Config	invalid FM/Product/Config
Optimisation	vaild Product/Config	invalid FM/Product/Config
Core feature	core feature	core feature
Variant feature	variant feature	variant feature
Dead feature detection	-	dead feature
False-optional feature detection	-	false-optional feature
Wrong-cardinality detection	-	wrong cardinal
Information Extraction Operation	**Configuration Explanatory Operations**	
	Why? operation	*Why not? operation*
Valid Configuration	valid partial conf.	invalid partial conf.

Cuadro A.1: Most frequently used explanatory operations and their corresponding information extraction operations

Bibliografía

[1] D. Benavides, A. Ruiz-Cortés, and P. Trinidad. Automated reasoning on feature models. *LNCS, Advanced Information Systems Engineering: 17th International Conference, CAiSE 2005*, 3520:491–503, 2005. ISSN 0302-9743. (pages 54, 56 y 60).